DATA FACT Z

Document Extraction System

Human-in-the-Loop Invoice Extraction Pipeline

Project Deliverables Report

DataFactZ AI Engineering Internship — Week 2, Use Case 2

Prepared by: Sudulagunta Brahmanya Asrit

Date: July 31, 2026

Contents

1. Problem Statement

Manual invoice data entry — retyping vendor names, line items, and totals from a PDF or scanned page into a system of record — is slow, repetitive, and error-prone at any real volume. It doesn't scale with headcount, and full manual review of every field on every document is the same cost whether the document was trivial or genuinely ambiguous.

At the same time, handing extraction entirely to an LLM with no oversight is not acceptable for financial documents: a model can misread a total, silently drop a line item, or hallucinate a field it doesn't actually see — and it will report this with the same apparent confidence as a correct extraction. A usable system has to combine the speed of automation with a trust mechanism that catches the cases the model got wrong, without forcing a human to re-check every field on every document regardless of risk.

Goal

Build a system that ingests both digital (text-layer) and scanned (image-only) invoices, extracts a fixed set of structured fields via an LLM, computes a genuine per-field confidence signal (not just the model's self-report), escalates only the documents that signal genuine uncertainty to a stronger model, and routes only the subset still uncertain after that to a human reviewer — so review effort is spent where it's actually needed.

Constraints

- Parsing/OCR must be open-source — no managed cloud document-AI services (Azure Document Intelligence, AWS Textract, Google Document AI). LLM APIs are allowed.
- Must handle both digital PDFs and scanned/photographed pages in the same pipeline.
- Corrections a human reviewer makes must be stored separately from the raw model output, so pre-review and post-review accuracy can both be measured from stored data — not just the final approved value.
- Accuracy comparisons must normalize formatting (dates, currency, whitespace) before exact-match comparison, so accuracy reflects real extraction errors, not formatting noise.

Success criteria

- Measurable field-level accuracy lift from raw model output to human-reviewed output on a held-out ground-truth set.
- Per-document processing time and per-document LLM cost, both observable from stored data rather than estimated after the fact.
- A working, demo-able system: upload → parse → extract → score → (maybe escalate) → review → export, with a branded UI throughout.

2. Solution Design Document

2.1 System Overview

The system is a FastAPI backend, a PostgreSQL database, and a React (TypeScript) frontend, with two external LLM deployments reached through a single Azure AI Foundry resource: `deepseek-v3.2` as the first-pass extraction model, and `gpt-5` as the escalation model for documents the first pass couldn't extract confidently. Both are OpenAI-SDK-compatible endpoints on the same resource, so the extraction service uses one client shape and differs only in which deployment name it calls. Scanned/image pages are parsed with PaddleOCR, running in its own worker process so a slow OCR call can't block the rest of the API.

> Every document — digital or scanned, high-confidence or low — passes through all five pipeline stages below. The design decision embedded throughout is *where* to spend extra model cost and human attention: only on the subset of documents that actually need it, decided by a measured confidence signal rather than a blanket policy.

2.2 Architecture Diagram

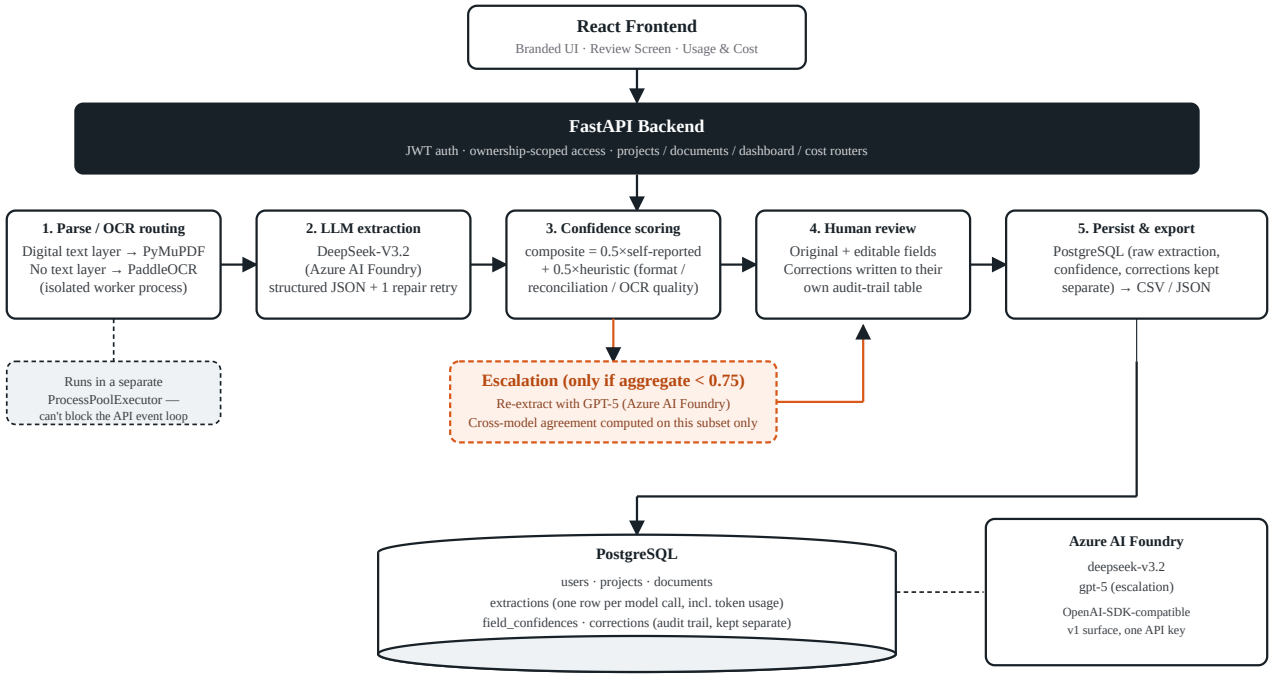

Figure 1 — End-to-end architecture: client, API, five-stage pipeline, and persistence.

2.3 Pipeline Walkthrough

Stage 1 **Parse / OCR routing** — for each page, PyMuPDF extracts the text layer directly. If fewer than 20 characters come back (a bare page number or a stray glyph shouldn't count as "has text"), the page is treated as scanned and rendered to a 200 DPI image for PaddleOCR instead — OCR accuracy drops sharply below roughly 150–200 DPI on real scans. This per-page routing is what lets one pipeline handle a clean digital invoice and a skewed phone-camera scan without a human pre-sorting them. PaddleOCR inference runs in a dedicated `ProcessPoolExecutor` (one worker, model loaded once), not the process serving the rest of the API — an earlier version ran OCR synchronously in-process and froze every other request (dashboard polling, document list, uploads) for the full duration of any image upload.

Stage 2 **LLM extraction** — DeepSeek-V3.2 runs against a Pydantic schema as its structured-output target. If the returned JSON fails to parse or fails schema validation, the pipeline sends one repair prompt quoting the exact validation error back to the model, rather than crashing the document or retrying unboundedly. A document that fails validation twice is marked `failed` and surfaced, not silently dropped.

Stage 3 **Confidence scoring & escalation** — each field's confidence is a composite, not the model's raw self-report: `composite = 0.5 × self_reported + 0.5 × heuristic_score`. The heuristic half comes from format/reconciliation checks specific to the field (e.g. `total_amount` checked against `subtotal + tax_amount`; line items checked against the invoice subtotal) and, for scanned pages, a multiplicative discount by the page's mean OCR confidence — a value pulled off a poorly scanned page is less trustworthy even if every other check passes. Fields marked `not_applicable` are excluded rather than penalized. If the document's aggregate composite falls below `confidence_escalation_threshold` (0.75), it is re-extracted with GPT-5; cross-model agreement between the two extractions is then computed *only* for that escalated subset, and a confirmed disagreement caps confidence regardless of either model's self-report.

Stage 4 **Human review** — every document, regardless of score, lands on a review screen with the original document next to editable extracted fields, tabbed by section (Overview, Parties, Amounts, Line Items for invoices). Confidence informs which fields get visually flagged as low-confidence, but never skips review outright. A reviewer can Approve (marks the document `approved`) or Flag for follow-up (saves the same corrections but leaves the document in `needs_review` for a second look). Corrections write to their own audit-trail table, keyed by field name and reviewer, distinct from the raw `Extraction` row(s) — so a report can compute accuracy against the raw model output and against the human-corrected output from the same stored history.

Stage 5 **Persistence & export** — every extraction call (DeepSeek pass, and the GPT-5 pass if escalated) is stored as its own row, including the prompt and completion token counts captured off the API response — the basis for the real, measured cost figures in Section 6, rather than an estimate. Approved and in-progress documents alike can be exported per document or per project as JSON or CSV; export uses the raw

extraction with each field's latest human correction layered on top, so what gets exported is the reviewed record, not the un-reviewed model output.

2.4 Data Model

The schema deliberately keeps four things separate rather than collapsing them into one "final record" table, because the accuracy report needs to compare raw-extraction accuracy against post-review accuracy from the same stored history:

TABLE	PURPOSE
`users`	Email, bcrypt password hash, role (`admin` / `user`). The first account to sign up is auto-promoted to admin; every account after that starts as a plain user and can only be promoted by an existing admin.
`projects`	A named batch of documents scoped to one document type (invoice / resume / contract). `owner_id` is nullable — projects created before ownership existed, or left behind by a deleted user, have no owner and are admin-visible only.
`documents`	One row per uploaded file: status, whether it was scanned, and per-stage timestamps (parsing/extraction started & completed) that back the dashboard's processing-time metrics.
`extractions`	One row per model call — the DeepSeek first pass, and the GPT-5 escalation pass if one happened — with the raw JSON result and its prompt/completion token counts. Never overwritten by a correction.
`field_confidences`	One row per scored field: self-reported score, heuristic score, composite, whether escalation happened, and cross-model agreement where applicable.
`corrections`	The human-review audit trail — field name, corrected value, reviewer, timestamp. Kept as its own table specifically so it never overwrites what the model actually produced.

2.5 API Surface

ENDPOINT	WHAT IT DOES
`POST /auth/signup` , `/auth/login`	JWT issuance; first account bootstraps as admin.
`GET/POST/DELETE /projects`	Project CRUD, scoped to the caller's own projects (admin also sees orphaned ones).
`POST /projects/{id}/documents`	Upload; queues `run_pipeline` as a background task and returns immediately.

`GET /documents/{id}`	Full detail: latest extraction, per-field confidence, aggregate score.
`POST /documents/{id}/corrections`	Writes correction rows and optionally marks the document approved.
`GET /documents/{id}/export`, `/projects/{id}/documents/export`	JSON/CSV export of the reviewed record, single document or whole project.
`GET /dashboard`	Aggregate stats — status counts, processing time, escalation rate, reviewed/auto-approved counts — optionally scoped to one project.
`GET /cost`	Per-project LLM cost for the caller; admin additionally gets a per-user breakdown and the total across every user.
`GET/PATCH/DELETE /users`	Admin-only user management; deleting a user cascade-deletes their projects, documents, and files.

2.6 Access Control & Multi-Tenancy

A single `core/ownership.py` module is the one place that decides who can see which project, reused by the projects, documents, dashboard, and cost routers instead of each duplicating an `owner_id` check: a regular user sees only their own projects; an admin sees their own plus any project orphaned by a deleted owner (never another active user's projects — admin is a role for user management, not blanket cross-tenant visibility). Document review (viewing, correcting, approving) follows the same rule — any user who can access the project can review its documents, not just admins, since ownership rather than role is what should gate review access.

2.7 Scalability & Deployment

Current deployment is a single FastAPI process, a single Postgres instance, and file storage on a local volume — sufficient for the project's scale. The explicit next steps at real scale, in the order they'd start to matter:

- **Background task queue.** Pipeline execution currently runs on FastAPI's in-process `BackgroundTasks`. That's fine for one instance; it doesn't survive a process restart and doesn't distribute across replicas. A Celery/Redis (or equivalent) queue is the natural next step once upload volume needs more than one API instance.
- **OCR worker pool.** Already isolated into its own process rather than the API's — the next step at scale is running that pool on separate, CPU-optimized instances behind the queue, since OCR is the CPU-bound stage and the LLM calls are I/O-bound.

- **Stateless API replicas behind a load balancer**, with Postgres connection pooling (e.g. PgBouncer) — the API itself holds no in-process state beyond the JWT it decodes per request, so this is a horizontal scale-out with no architecture change.
- **File storage** moves from a local volume to object storage (S3-compatible) once more than one instance needs to serve the same uploaded file.

2.8 Security

- Passwords hashed with bcrypt; sessions are stateless JWTs (HS256, 7-day expiry) validated on every request via a FastAPI dependency, not a server-side session store.
- Role is never client-supplied at signup — it's derived server-side (first account only) and changed only by an existing admin, so nobody can self-assign admin.
- Every project/document endpoint re-checks ownership server-side (see 2.6) — the frontend hiding a nav link is a UX nicety, not the access control.
- The one endpoint that accepts a JWT as a query parameter instead of a header (`GET /documents/{id}/file`) exists only because an `<img>`/PDF viewer element can't attach an Authorization header — still validated the same way.

3. Pattern Justification

Each major design decision below records at least one alternative that was considered and rejected, and why — including two cases where the rejected alternative was actually built first and then replaced once its consequence showed up in practice.

DECISION	CHOSEN APPROACH	REJECTED ALTERNATIVE(S) & WHY
Parsing strategy	Per-page routing: PyMuPDF for pages with a text layer, PaddleOCR for pages without.	**A single fixed parser** (always OCR, or always text-extraction) — fails outright on whichever input type it wasn't built for. **Managed cloud document-AI** (Textract / Document AI / Azure Document Intelligence) — excluded by the project's own constraint, and would lock the pipeline to one vendor's pricing and availability.
Extraction retry	Validate → on failure, one repair prompt quoting the exact validation error → fail the document cleanly if that also fails.	**No retry** — a single transient bad JSON response fails documents that would have succeeded on a second try. **Unbounded retry** — risks looping on a genuinely-impossible extraction and burning cost with no completion guarantee.
Escalation model tier	Two-tier: DeepSeek on every document; GPT-5 only on the subset below the confidence threshold.	**Always run both models on every document** — doubles LLM cost on every document with no escalation savings, since cross-model agreement would then be computed (and paid for) even on documents already extracted with high confidence.
Confidence signal	Composite score: half the model's self-reported confidence, half independent heuristic checks (format validity, cross-field reconciliation, OCR quality).	**Raw self-reported LLM confidence alone** — models are frequently overconfident or miscalibrated on their own output; a model can report high confidence on a field it misread.
Review gating	Every document requires a human review step, regardless of confidence score.	**Confidence-based auto-approval** above threshold — considered and rejected; confidence informs escalation and which fields get flagged for scrutiny, but a composite score is a heuristic, not a guarantee, and this is a financial-document pipeline where an unreviewed wrong total is a worse failure mode than one extra minute of review time.
Correction storage	Corrections write to their own audit-trail table, keyed by field and reviewer.	**Overwrite the raw extraction in place** — would make it impossible to compute raw-model accuracy

		vs. post-review accuracy from the same stored data, which the project's own accuracy report requires.
Deleted-user data handling	Deleting a user cascade-deletes their projects, documents, and files.	**Orphan the user's projects onto admin** (owner_id set to NULL, admin-visible) — this was actually shipped first, then replaced: it meant deleting a user handed their reviewed documents to admin by default, which is a privacy regression, not a feature, once actually used. Orphaning is still kept for the unrelated pre-ownership legacy-data case.
OCR execution	PaddleOCR inference runs in a dedicated `ProcessPoolExecutor`, isolated from the API process.	**Synchronous in-process OCR** (even inside a background task) — this was the original implementation, and it froze every other request (dashboard, polling, unrelated uploads) for the full duration of any scanned-document upload, since it pinned the single process's CPU/GIL time.
Background job execution	FastAPI's in-process `BackgroundTasks` for the pipeline.	**A dedicated job queue** (Celery + Redis/RabbitMQ) — the more scalable choice, deliberately deferred rather than rejected outright: premature at single-instance scale, and called out explicitly in Section 2.7 as the first thing to add when the API needs to run more than one replica.

4. Working Demo

The screenshots below are from a live run against the deployed system, not mockups: a fresh account signed up, a project created, and two real invoices from the project's test set uploaded and run through the full pipeline — one clean digital PDF, and one scanned page that genuinely came back too degraded to extract, demonstrating the escalation path actually triggering and the pipeline's honest failure handling rather than a cherry-picked success case.

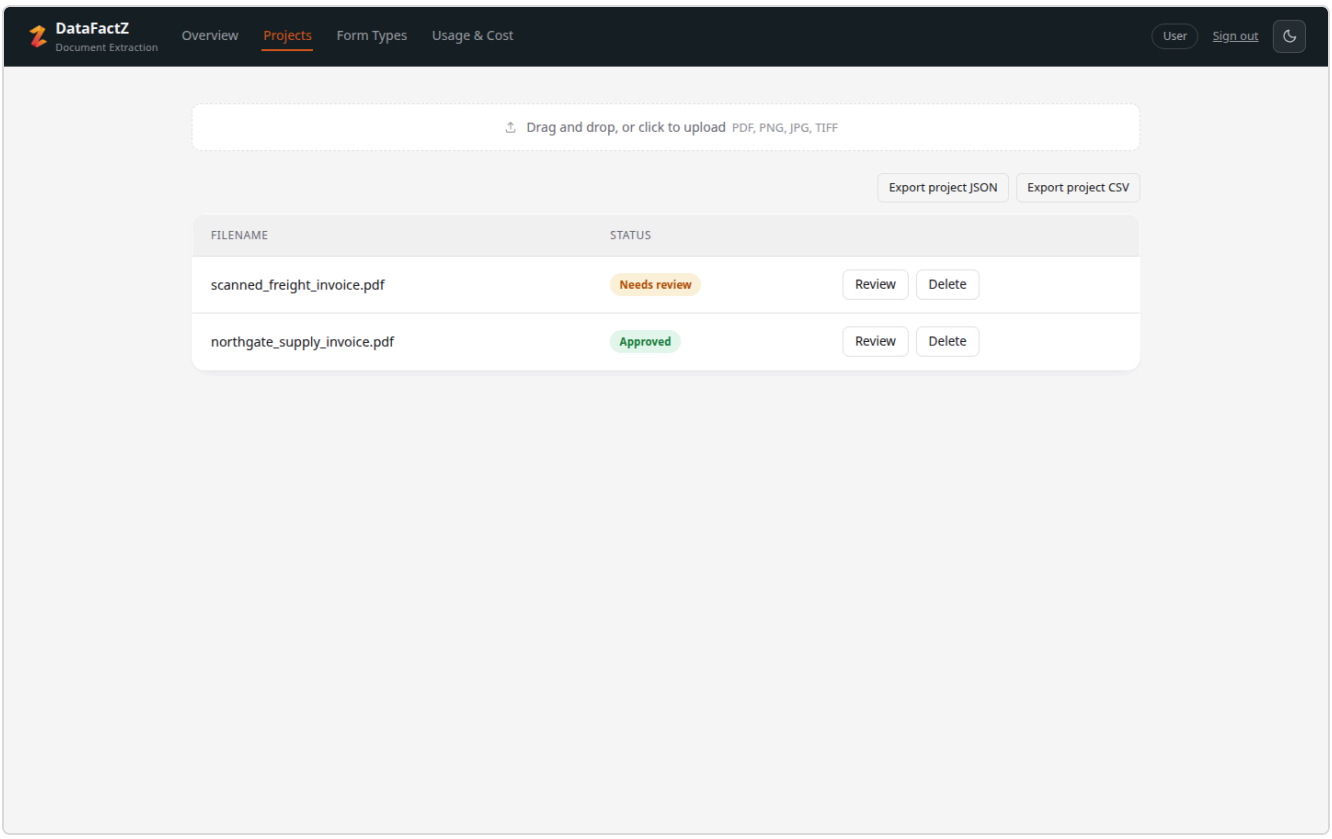

Figure 2 — Project queue after both uploads finished processing: one auto-escalated scanned document still needing review, one digital document already approved. Bulk JSON/CSV export for the whole project sits above the queue.

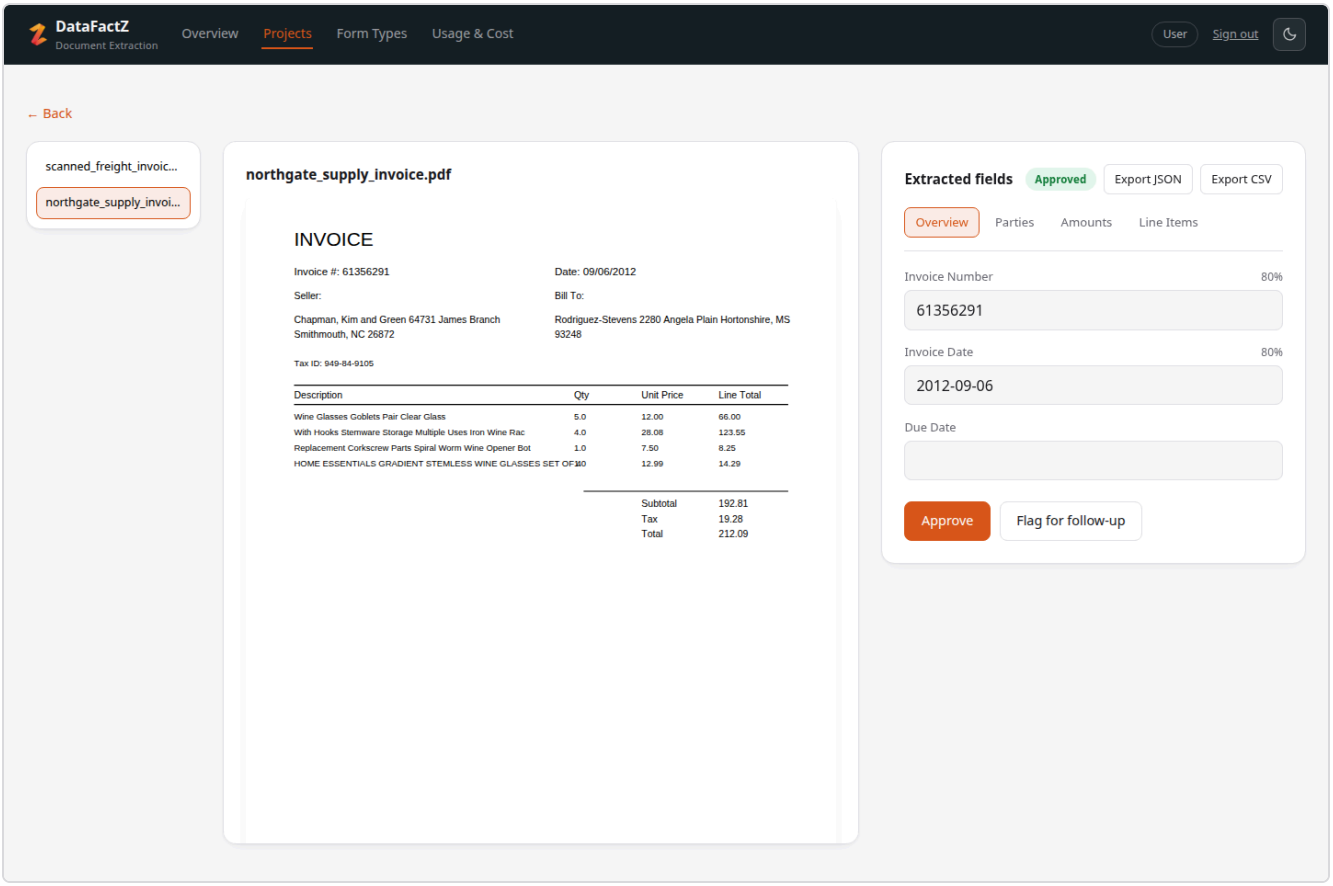

Figure 3 — Review screen for the clean digital invoice: DeepSeek extracted every field with an 80–100% composite score on the first pass, no escalation needed. Per-document Export JSON/CSV sits above the extracted fields.

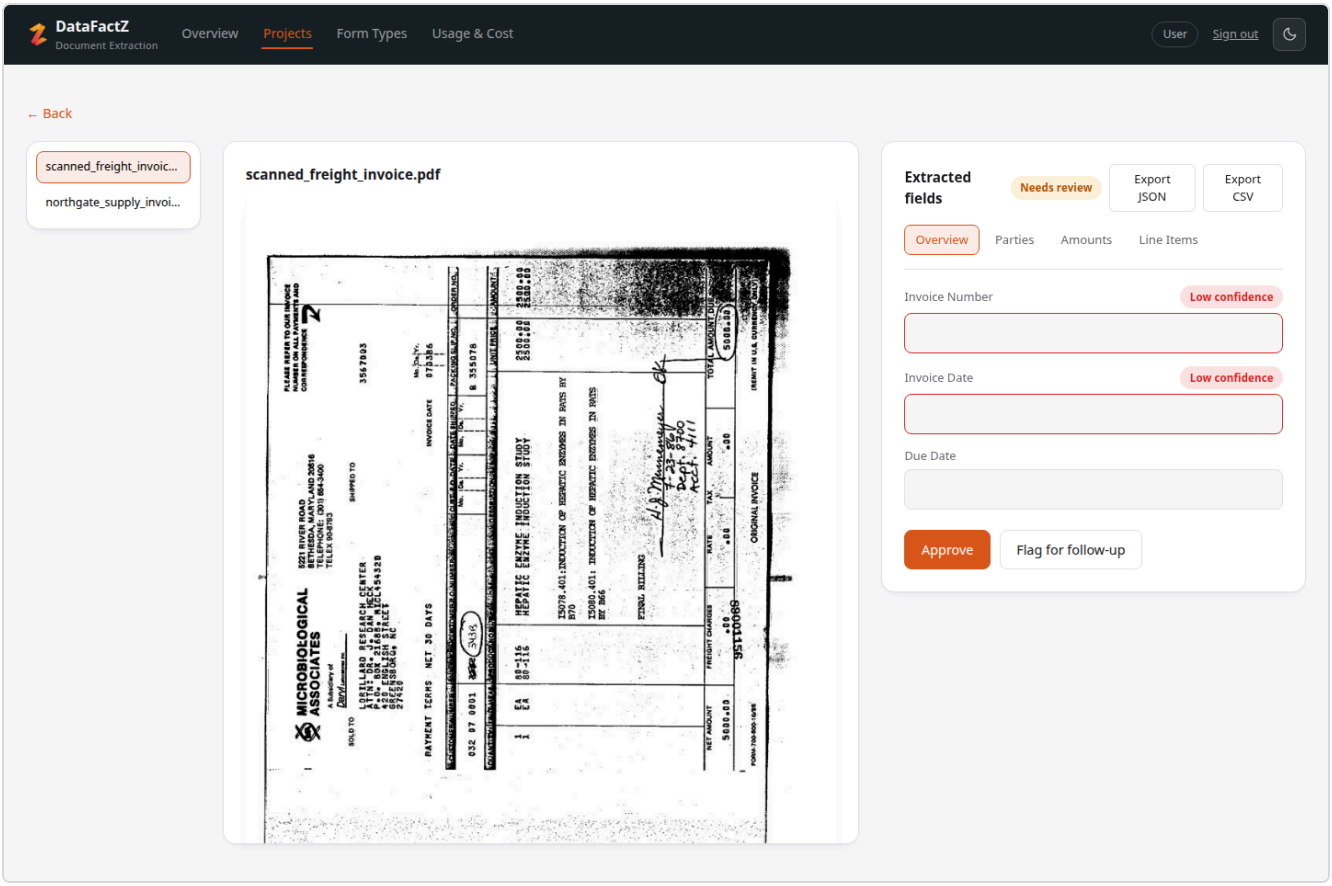

Figure 4 — The scanned invoice after escalation: the page came back genuinely illegible even to GPT-5, so every field's `field_status` is `illegible` and the composite score bottoms out at 0.25 — exactly the case the pipeline is designed to route to a human rather than guess at.

Real measured outcome for this run: document 72 (digital) extracted at aggregate confidence 0.82 with no escalation; document 73 (scanned) escalated, and both extractions' prompt/completion token counts were captured and used to compute the real cost figures in Section 6 — `1,148/467` tokens for the DeepSeek pass on document 72, and `1,215/251` then `1,189/3,763` for the DeepSeek and GPT-5 passes on document 73.

5. Branded UI

The interface follows DataFactZ's brand palette throughout — navy header, orange accent for active state and primary actions, and the DataFactZ mark in the top-left of every authenticated page — rather than a generic component-library default theme. Both light and dark themes are supported (the screenshots below are light theme); the palette below is defined once as CSS custom properties and referenced everywhere rather than hardcoded per component.

Brand Navy	Accent Orange	Surface	Text
#182127	#d9591a	#f7f7f8	#1c1c1f

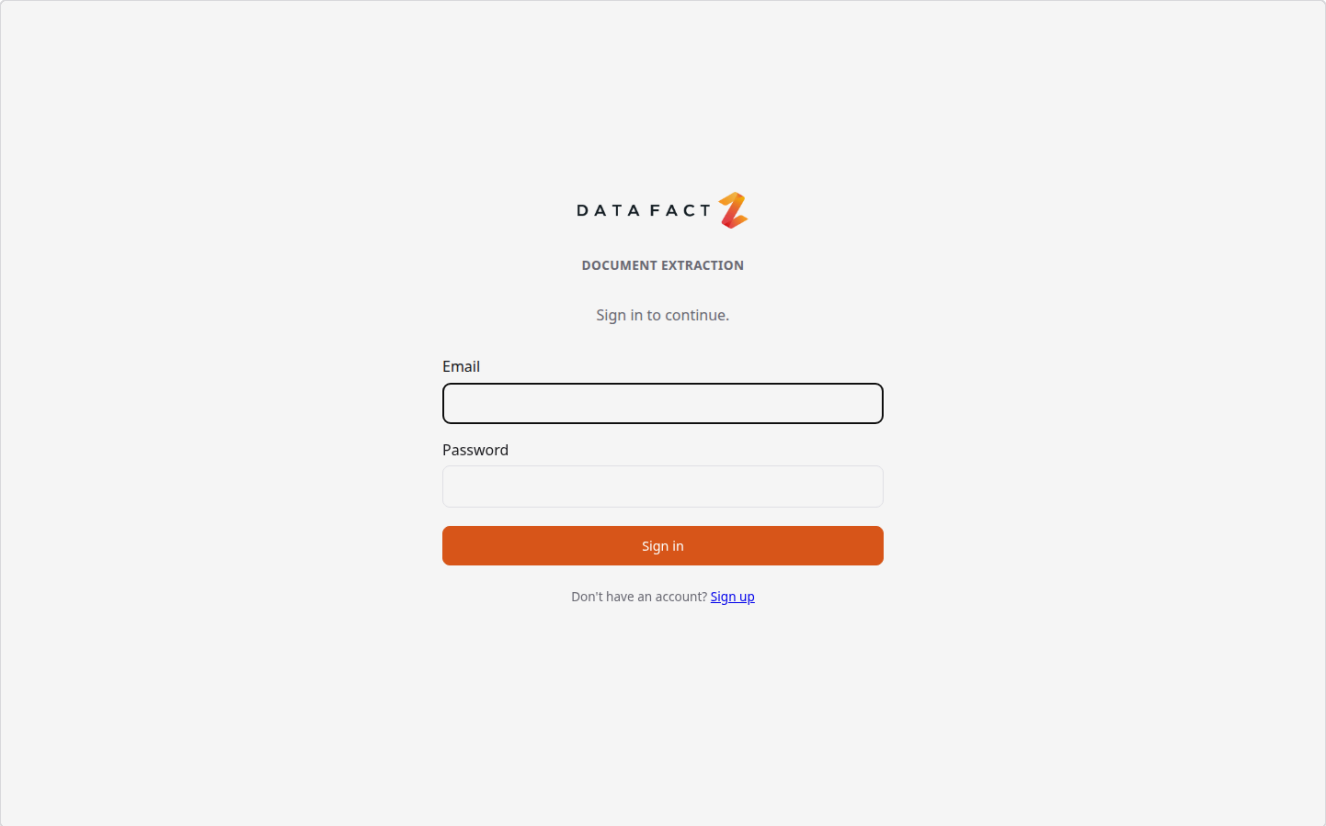

Figure 5 — Sign-in screen: DataFactZ wordmark, tagline, minimal branded form.

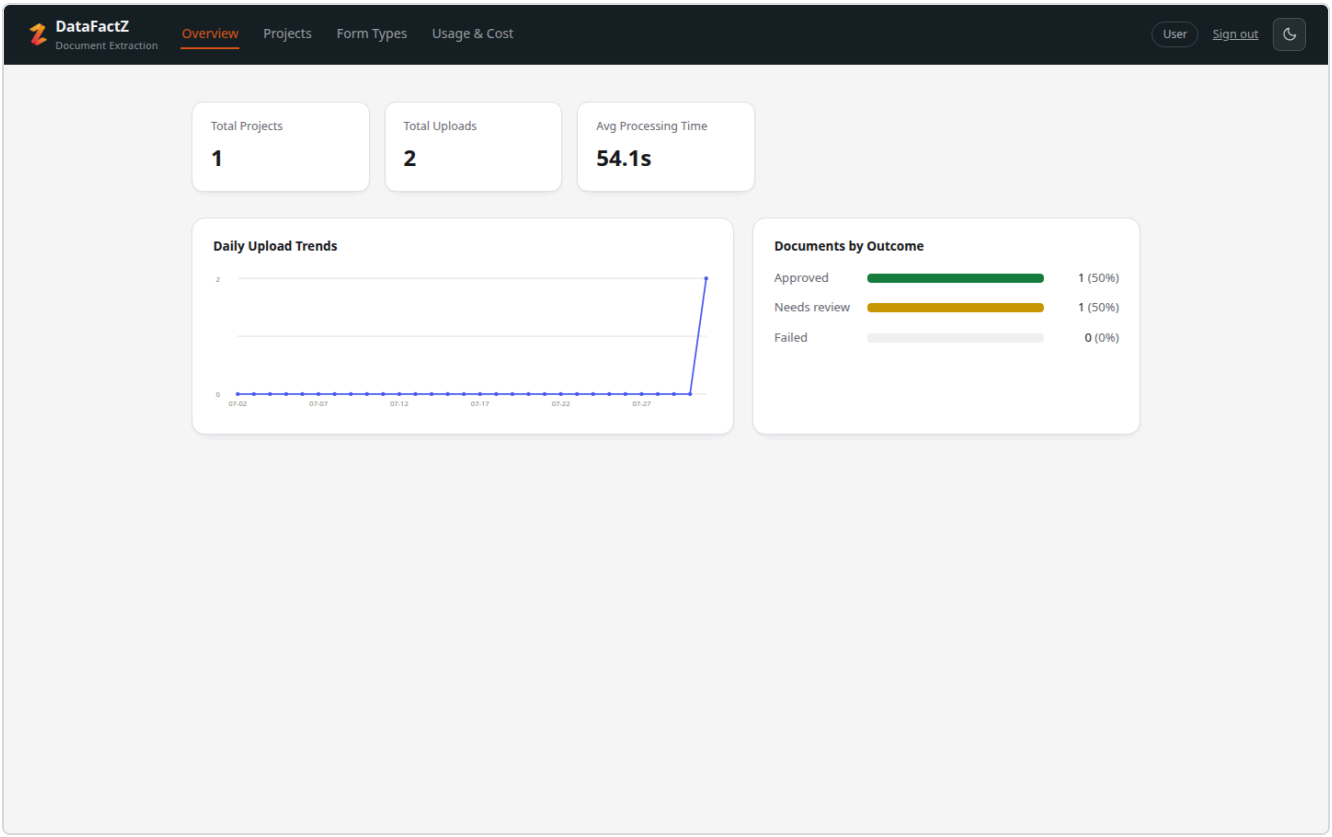

Figure 6 — Overview: global stats and trend charts, the landing page after sign-in.

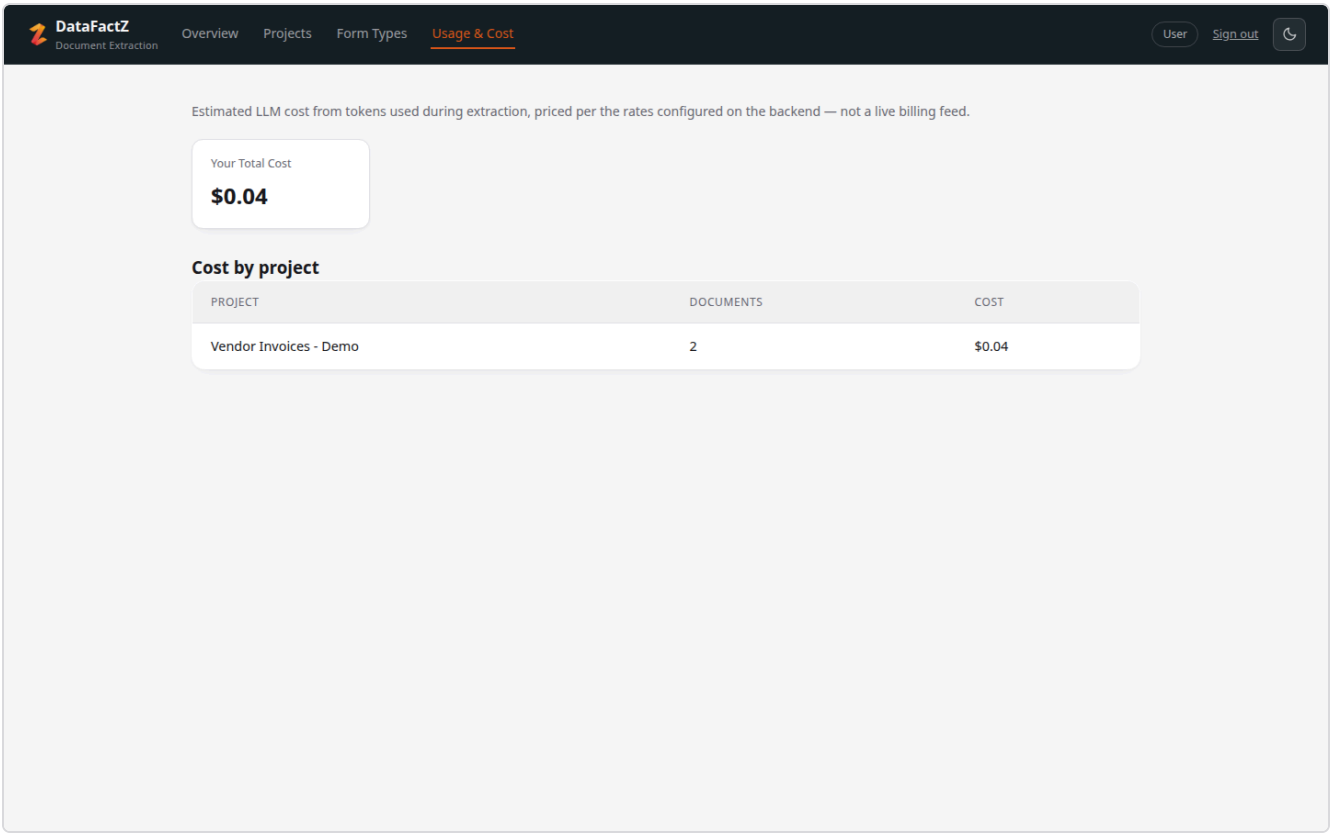

Figure 7 — Usage & Cost: per-project LLM cost, computed from real captured token usage (Section 6).

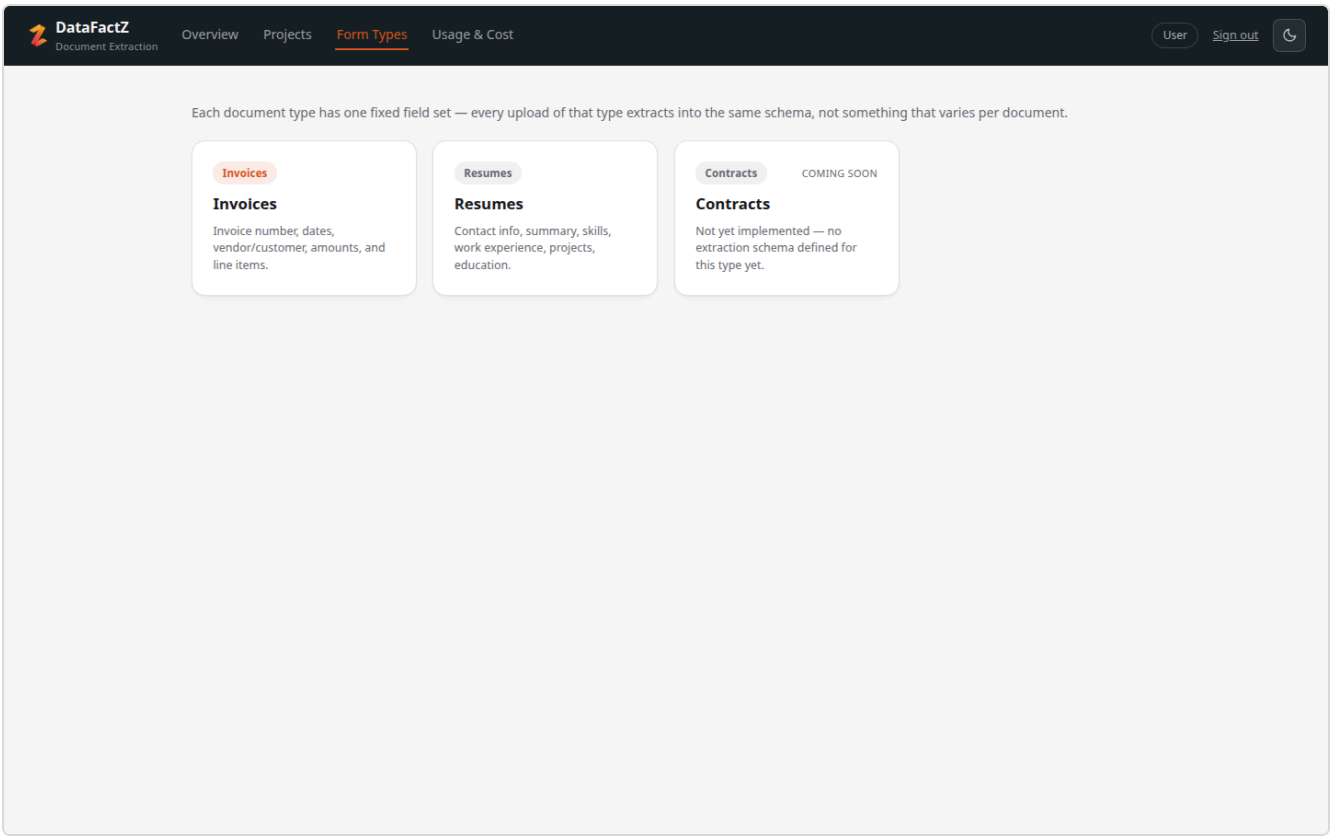

Figure 8 — Form Types: the fixed field set per document type (invoice implemented; resume implemented; contract flagged "coming soon" rather than silently mis-extracting).

6. Cost Estimate

These figures are built from real measured token usage on this deployment (Section 4), not a hypothetical per-call estimate — `extraction.py` captures the `usage.prompt_tokens` / `usage.completion_tokens` off every API response, and the cost service prices them at each deployment's published Azure AI Foundry rate.

MODEL	$ / 1M INPUT TOKENS	$ / 1M OUTPUT TOKENS
DeepSeek-V3.2 (first pass, every document)	$0.58	$1.68
GPT-5 (escalation only)	$1.25	$10.00

CASE	TOKENS (PROMPT / COMPLETION)	MEASURED COST
Digital invoice, no escalation (document 72)	1,148 / 467	$0.0015
Scanned invoice, escalated — DeepSeek pass (document 73)	1,215 / 251	$0.0011
Scanned invoice, escalated — GPT-5 pass (document 73)	1,189 / 3,763	$0.0391
Document 73 total (both passes)	—	**$0.0402**

An escalated document costs roughly **28×** a non-escalated one in this sample — almost entirely from GPT-5's output-token rate, which is why the escalation gate (Section 2.3) matters for cost, not just accuracy: escalating every document instead of only the uncertain subset would multiply the whole pipeline's LLM cost by roughly that factor.

Stated assumption, since the brief doesn't fix a volume: 20 documents per active user per month. Blended per-document cost = (1−escalation rate) × digital cost + escalation rate × escalated cost, at three illustrative escalation rates.

At 100 users

ESCALATION RATE	BLENDED $/DOCUMENT	ESTIMATED MONTHLY LLM COST
15%	$0.0073	$14.54
30%	$0.0131	$26.18
40%	$0.0170	$33.93

At 5,000 users

ESCALATION RATE	BLENDED $/DOCUMENT	ESTIMATED MONTHLY LLM COST
15%	$0.0073	$726.92
30%	$0.0131	$1,308.81
40%	$0.0170	$1,696.73

Infrastructure (illustrative, provider-agnostic)

COMPONENT	~100 USERS	~5,000 USERS
API compute (FastAPI, stateless)	1 small instance, ~$25–40/mo	Several instances behind a load balancer, ~$300–500/mo
OCR worker compute	Shared with API instance	Dedicated CPU-optimized pool, ~$200–400/mo
PostgreSQL	Small managed instance, ~$15–30/mo	Larger managed instance + pooling, ~$150–300/mo
File storage	Local/small volume, ~$5/mo	Object storage, ~$50–100/mo at volume
Infra subtotal	**~$45–75/mo**	**~$700–1,300/mo**

Infrastructure figures are order-of-magnitude, not a vendor quote — no specific cloud provider or instance size has been committed to; they're included to show the LLM cost is the smaller line item at this scale, not the dominant one, until volume grows well past 5,000 users.

7. AI Usage Log

This entire project was built with Claude Code (Anthropic's agentic coding CLI) as a pair-programming tool throughout — design discussion, implementation, debugging, and this report itself. The table below is the real commit history, not a reconstructed summary; every commit below carries a `Co-Authored-By: Claude Sonnet 5` trailer.

DATE	WORK	NATURE OF AI INVOLVEMENT
2026-07-27	Initial repo setup	Project scaffolding discussion
2026-07-28	Implement invoice extraction pipeline end to end	Full pipeline implementation: parsing/OCR routing, DeepSeek extraction, confidence scoring, review UI, from a conceptual discussion of the design
2026-07-28	Generalize pipeline to multi-document-type, add dashboard, fix admin bootstrap	Refactor + new feature implementation
2026-07-28	Add per-user data ownership, Foundry model config, and review document sidebar	Multi-tenancy design + implementation
2026-07-28	Apply DataFactZ brand identity to UI	Branding pass across the frontend
2026-07-29	Remove redundant "Original document" heading; remove auto-derived ground_truth files	UI cleanup and repo hygiene, prompted by review
2026-07-30	Isolate OCR into a separate worker process, fix missing loading states	Diagnosed a real production-shaped bug (OCR blocking the event loop) and fixed it, plus two related UX gaps found during the same investigation
2026-07-30	Trim invoice schema to core fields	Schema simplification via the project's schema-change skill
2026-07-30	Orphan a deleted user's projects instead of leaving a dangling owner	Data-integrity design decision, later revisited (see next entry and Section 3)

2026-07-31	Add per-project cost tracking, document export, and ownership-scoped access	New feature build: token-usage capture, cost service, JSON/CSV export endpoints; also fixed a schema-validation bug (resume Skills corrections always failing), a delete-user foreign-key ordering bug, and reversed the previous day's orphan-on-delete decision to cascade-delete instead, after discussing the privacy implication
2026-07-31	Redesign top-level navigation, add Usage & Cost page, fix review-flow bugs	Navigation restructuring per user direction, new page implementation, and two independently-diagnosed bugs (Back button history-stack behavior, resume Skills 422)
2026-07-31	Dead-file audit	Systematic check for unused frontend assets and components (import-graph search), confirming six unused files (Vite scaffold defaults and unused favicon/logo variants)
2026-07-31	This deliverables report	Content authored from the actual codebase and git history (not templated placeholder text); screenshots captured live against the running system via a scripted browser session; cost figures computed from real token counts queried from the database, not invented for the report

Model used throughout: Claude Sonnet 5, via Claude Code. Every architectural decision above was discussed and confirmed before implementation — the AI's role was pair-programmer and implementer, not an unsupervised autonomous author.